



by

Kashaf Fatima 22i-2415

Safi ur Rehman 22i-2534

Amna Noor 22i-1529

Software Reengineering Reengineering Plan

Submitted to Nigar Azhar Butt

Date: 07/12/2025

Contents

1 Introduction	3
2 Phase 1: Inventory Analysis	3
2.1 Legacy System Overview	3
2.2 Component Inventory	3
2.3 Identified Limitations	3
3 Phase 2: Reverse Engineering	4
3.1 Data Structure Recovery	4
3.2 Business Logic Extraction	4
4 Phase 3: Restructuring (Architecture Design)	4
4.1 Target Architecture: MVC on MEN Stack	4
4.2 New Schema Design (MongoDB)	5
5 Phase 4: Forward Engineering	5
5.1 Implementation Details	5
6 Phase 5: Migration Strategy	6
6.1 Data Migration (ETL Pipeline)	6
6.2 Cutover Plan	6
7 Timeline	6
8 Risk Considerations	6
9 Conclusion	7

1 Introduction

This document outlines the comprehensive reengineering and migration plan for the legacy Point of Sale (POS) system. The objective is to transition from a monolithic, file-based Java Swing application to a modern, scalable, web-based architecture using the Node.js, Express, and MongoDB (MEN) stack. This transformation aims to address critical limitations in scalability, security, and maintainability found in the legacy system.

2 Phase 1: Inventory Analysis

The first phase involves a detailed inventory of the existing legacy system to understand its scope, components, and dependencies.

2.1 Legacy System Overview

- **Project Name:** Point-of-Sale-System-master
- **Language:** Java (JDK 8+)
- **UI Framework:** Java Swing (JFrame, JButton, JOptionPane)
- **Data Storage:** Flat Text Files (.txt)
- **Build System:** Apache Ant (build.xml) / Eclipse Project

2.2 Component Inventory

Component Type	Count	Key Files
User Interface	12 Classes	Login _Interface.java, Payment Interface.java
Business Logic	8 Classes	POSSystem.java, Management.java, POS.java
Data Models	5 Classes	Employee.java, Item.java, ReturnItem.java
Data Files	4 Files	userDatabase.txt, itemDatabase.txt

Table 1: Legacy System Component Inventory

2.3 Identified Limitations

1. **Scalability:** Text file storage ($O(N)$ complexity) is unsuitable for high transaction volumes.
2. **Concurrency:** No locking mechanism leads to data corruption in multi-user scenarios.
3. **Security:** Plain text passwords and lack of payment gateway integration.

4. **Maintainability:** Tight coupling between UI and logic makes testing and updates difficult.

3 Phase 2: Reverse Engineering

In this phase, we analyzed the legacy source code to extract business rules and data structures.

3.1 Data Structure Recovery

By analyzing Management.java and POSSystem.java, we reconstructed the implicit schema:

- **User/Customer:** Phone Number (PK), Rental History.
- **Item/Inventory:** Item ID, Name, Price, Quantity.
- **Employee:** ID, Name, Username, Password, Role (Admin/Cashier).
- **Transaction:** Linked via temporary session files and direct updates to inventory text files.

3.2 Business Logic Extraction

Key workflows identified:

- **Login:** Direct string comparison of username/password against employeeDatabase.txt.
- **Sale:** Decrement item quantity in itemDatabase.txt upon "End Transaction".
- **Rental:** Append rental record to userDatabase.txt with "false" (not returned) status.
- **Return:** Parse userDatabase.txt, calculate late fees based on date difference, update status to "true".

4 Phase 3: Restructuring (Architecture Design)

The restructuring phase defines the blueprint for the new ModernPOS system.

4.1 Target Architecture: MVC on MEN Stack

We are moving to a Layered Architecture:

- **Frontend (View):** HTML5, CSS3, Client-side JavaScript (served via public folder).
- **Backend (Controller):** Node.js with Express.js REST API.

- **Database (Model):** MongoDB (NoSQL) with Mongoose ODM.

4.2 New Schema Design (MongoDB)

```
// User Schema
{
  phone: String, rentals: [{
    itemId: Number, returnDate:
    Date, isReturned: Boolean
  }]
}
// Item Schema
{
  itemId: Number, itemName:
  String, price: Number, amount:
  Number
}
// Employee Schema
{ username: String, password: String, // Hashed in future
  role: String
}
```

5 Phase 4: Forward Engineering

This phase involves the actual implementation of the ModernPOS system based on the restructured design.

5.1 Implementation Details

- **Server Setup:** server.js initializes Express, connects to MongoDB via Mongoose, and serves static assets.
- **API Routes:** routes/api.js handles all CRUD operations:
 - POST /api/login: Authenticates employees.
 - GET /api/items: Retrieves inventory.
 - POST /api/rent: Handles rental transactions transactionally.
 - POST /api/return: Processes returns and updates inventory.
- **Frontend:** dashboard.html consumes these APIs using fetch() to dynamically update the UI without page reloads.

6 Phase 5: Migration Strategy

A "Big Bang" migration is risky. We propose a Phased Migration strategy.

6.1 Data Migration (ETL Pipeline)

Since the source is text files and the destination is MongoDB, we need a custom ETL script.

Step 1: Extract Read .txt files line-by-line using Node.js fs module.
`fs.readFileSync('Database/itemDatabase.txt', 'utf-8').split('\n');`

Step 2: Transform Parse the delimiter-separated values (spaces or commas) into JSON objects. Convert data types (String to Number for prices). **Step 3: Load** Use Mongoose insertMany() to bulk load the transformed data into MongoDB collections.

6.2 Cutover Plan

1. **Freeze Legacy System:** Stop all writes to the Java system (after business hours).
2. **Run Migration Script:** Execute the ETL script to populate MongoDB.
3. **Verify Data:** Random sampling of 10% of records to ensure accuracy.
4. **Launch ModernPOS:** Start the Node.js server.
5. **Decommission:** Archive legacy text files.

7 Timeline

Phase	Duration	Deliverables
Inventory & Analysis	1 Day	Asset list, Limitation Report
Reverse Engineering	1 Day	Logic extraction, Schema mapping
Forward Engineering	3 Days	Functional MVP (API + UI)
Data Migration	3 Days	ETL Scripts, Populated DB
Testing & Training	1 Day	UAT Sign-off, User Manuals
Total	6-7 Weeks	

8 Risk Considerations

- **Data Loss:** Risk of parsing errors during ETL due to inconsistent text file formatting. *Mitigation:* Write strict validation logic in the Transform step and log all rejected lines.
- **Downtime:** Extended downtime during cutover. *Mitigation:* Perform a dry-run migration on a staging environment first.

- **User Resistance:** Staff accustomed to the Swing UI may struggle with the Web UI.
Mitigation: Conduct training sessions and keep the UI layout familiar initially.

9 Conclusion

The migration from the legacy Java POS to ModernPOS represents a significant leap in technology and capability. By moving to a web-based, database-backed architecture, the system becomes scalable, secure, and ready for future integrations (e.g., cloud deployment, mobile access). The outlined plan ensures a structured, low-risk transition.